Erweitern und Festigen der erlernten Konzepte

31.10.2025

Plan für heute

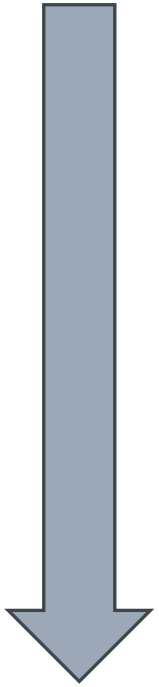
- Wiederholung Parentheses & Precedence
- Ternäre Operator (ternary)
- switch

Parentheses & Precedence

WIEDERHOLUNG DER KONZEPTE

Precedence

Absteigende Prio



Operatoren
()
++, - -
*, /, %
+, -
<, >, <=, >=, ==, !=
&, |
&&, ||
= , +=, -=, …

Parentheses (Klammern)

- Klammern haben die höchste Priorität und bestimmen die Reihenfolge der Berechnung
- können genutzt werden, um komplizierte Ausdrücke verständlicher zu machen
- Beispiel:

```
int result = (2+3) * 4;        //Ergebnis: 20

int result = 2 + 3 * 4;        //Ergebnis: 14
```

Was ist die Ausgabe?

```
int result = 3 + 2 * 5 - (7 + 2) * (4 - 1);
```

Was ist die Ausgabe?

```
int result = 3 + 2 * 5 - (7 + 2) * (4 - 1); //Klammern
int result = 3 + 2 * 5 - 9 * 3;  //*
int result = 3 + 10 – 27
int result = -14;
```

Was ist die Ausgabe?

```
int result = (4 + 3) * (5 - 2) / 3 + 6;
```

Lösung: 13

Was ist die Ausgabe?

```
int a = 5, b = 10;
boolean result = (a++ * b-- + (a * b)) > a && (a < b || b > 15);
```

Was ist die Ausgabe?

```
int a = 5, b = 10;
boolean result = (a++ * b-- + (a * b)) > a && (a < b || b > 15); //erste Klammer

boolean result = (5 * 10 + (a * b)) > a && (a < b || b > 15);  //innere Klammer mit neuen Werten

boolean result = (50 + 6 * 9) > a && (a < b || b > 15);

boolean result = 50+ 54 > a && (a < b || b > 15);

boolean result = 104 > a && (a < b || b > 15);

boolean result = true && true;
```

Was ist die Ausgabe?

```
int x = 4, y = 3;
boolean result = (x++ > y || –y > 2) && (x == 5);
```

Was ist die Ausgabe?

```
int x = 4, y = 3;
boolean result = (x++ > y || –y > 2) && (x == 5);  //Klammern

boolean result = (4 > 3 || 2 > 2) && (5 == 5);       //Klammern

boolean result = true && true;
```

Was ist die Ausgabe?

```
int x = 6, y = 8;
boolean condition1 = (++x > y--) && (y == 8);
boolean condition2 = (x++ < --y) || (++x == 7);
boolean result = condition1 || condition2;
```

Lösung: true

Was ist die Ausgabe?

```
int i = 1, j = 2;
boolean result = (++i > 1) & (–j > 0) | (i == 2 && j-- == 1);
```

Was ist die Ausgabe?

```
int i = 1, j = 2;
boolean result = (++i > 1) & (–j > 0) | (i == 2 && j-- == 1); //Klammern

boolean result = (2 > 1) & (1 > 0) | (i == 2 && j– == 1); //Klammern

boolean result = (true) & (true) | (true && true);

boolean result = true | true;

boolean result = true;
```

Ternärer Operator

WIEDERHOLUNG DER KONZEPTE

Ternärer Operator

- mit dem ternären Operator kann man if-Bedingungen umschreiben
- Syntax:

```
variable = (Bedingung) ? Ausdruck1 : Ausdruck2;
```

Ternärer Operator

– Beispiel

```
int a = 5;
int b;

if (a > 3) {
    b = 10;
} else {
    b = 20;
}
```

=

```
int a = 5;
int b = (a > 3) ? 10 : 20;
```

wenn true: 10

wenn false: 20

Ternärer Operator

– Beispiel

```
if(a > 3){
    result = 10;
}else if(b > 6){
    result = 15;
}else{
    result = 30;
}
```

=

```
result = (a>3) ? 10 : (b>6) ? 15 : 30;
```

wenn b > 6 true

Wenn if und elseif falsch 30

wenn true: 10

wenn if false: b > 6?

Komplexe Ternärer Operator

– Beispiel

```
if (a > 5) {
    if (b > 6) {
        result = 10;
    } else {
        result = 20;
    }
} else {
    result = 30;
}
```

=

```
int a = 3;
int b = 5;
int result = (a > 5) ? ((b > 6) ? 10 : 20) : 30;
```

Komplexe Ternärer Operator

– Beispiel

```
int a = 3;
int b = 5;
int result = 0;

if(a> 5){
    if(b > 6){
        result = 10;
    }
}else{
    result = 20;
}
```

=

```
int a = 3;
int b = 5;
int result = 0;

result = (a > 5)? ((b > 6) ? 10 : result) : 20;
```

Komplexe Ternärer Operator

– Beispiel

```
int a = 3;
int b = 5;
int result = 0;

if(a> 5){
    if(b > 6){
        result = 10;
    }
}else{
    result = 20;
}
```

Würde b > 6 nicht zutreffen, wäre result immer noch 0

=

```
int a = 3;
int b = 5;
int result = 0;

result = (a > 5)? ((b > 6) ? 10 : result) : 20;
```

Deswegen wird result der ursprüngliche Wert (=0) wieder zugewiesen

Übung ca. 20 Minuten

Überprüfe, ob eine Zahl (num) positiv, negativ oder null ist, und gib eine entsprechende Nachricht aus.

- Positiv: Ausgabe: "Die Zahl ist positiv."
- Negativ: Ausgabe: "Die Zahl ist negativ."
- Null: Ausgabe: "Die Zahl ist null.„

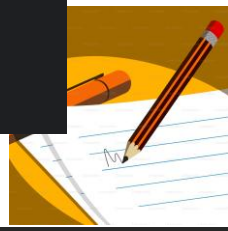
Schreibe diese Bedingung in einem if-else Konstruktion. Schreibe deine Bedingung in einen ternary um.

Übung ca. 10 Minuten

- Versucht das in einen ternary umzuschreiben

```
int age = 25;
boolean isStudent = true;
boolean hasJob = false;
String result = „“;


if (age < 18) {
   result = "Du bist noch ein Jugendlicher.";
} else if (age >= 18 && age <= 30) {
   if (isStudent) {
      result = "Du bist ein Student.";
   } else {
      result = "Du bist ein junger Erwachsener.";
   }
} else {
      result = "Du bist ein erfahrener Erwachsener.";
}
```



```
int age = 25;
boolean isStudent = true;
boolean hasJob = false;
String result = „“;


if (age < 18) {
    result = "Du bist noch ein Jugendlicher.";
} else if (age >= 18 && age <= 30) {
    if (isStudent) {
        result = "Du bist ein Student.";
    } else {
        result = "Du bist ein junger Erwachsener.";
    }
} else {
        result = "Du bist ein erfahrener Erwachsener.";
}
```

```
String result =
(age < 18) ? "Du bist noch ein Jugendlicher." :
(age >= 18 && age <= 30) ? (isStudent ? "Du bist ein Student." : "Du bist ein junger Erwachsener.") : "Du bist ein erfahrener Erwachsener.";
```

Arrays

WIEDERHOLUNG DER KONZEPTE

Initialisierung

Size **MUSS** hinten stehen

C-Style.
Möglich, aber
vermeiden

Illegal!

```
int[] i = new int[5];
int i2[] = new int[5];
int[5] i3 = new int[];      //Compilerfehler
int[][] i4 = new int[2][3];
int[][] i5 = new int[2][];
int[] i8[] = new int[2][3];
int[][5] i6 = new int[][];  //Compilerfehler
int[5][5] i7 = new int[][]; //Compilerfehler
```

Unschön, geht
aber

Grundlagen

- Arrays sind Objekte!

- wenn man also ein Array nicht initialisiert, ist der Standardwert null!

- 1-D-Array ist ein Konstrukt, in welchem einzelne Elemente gespeichert werden

```
int[] i = new int[] {1,2,3,4,5,6};
```

- 2-D-Array ist ein Konstruktor, in welchem mehrere Arrays gespeichert werden

```
int[][] i = new int[][] {{1,2,3}, {1,2,3}, {1,2,3,4,5,6}};
```

Grundlagen

- in Arrays stehen default-Werte
  - Bei Arrays mit primitiven Elementen (Bsp: int -> 0, double -> 0.0, ...)

```
int[] iArray = new int[5];
for(int i: iArray){
   System.out.print(i);
}
```

Ausgabe: 00000

  - Bei Arrays mit Objekten (Bsp. String -> null)

```
String[] sArray = new String[5];
for(String i: sArray){
   System.out.print(i);
}
```

Ausgabe: nullnullnullnullnull

Elemente setzen

- das Setzen von Elementen erfolgt über den Index

- das erste Element hat den Index 0

- Bsp. 1-D- Array:

```
String[] sArray = new String[5];
sArray[0] = "Hello";
sArray[1] = "My";
sArray[2] = "My";
sArray[3] = "My";
sArray[4] = "My";
```

Elemente setzen

- Bsp. 2-D- Array:

```
String[][] s2Array = new String[5][3];
s2Array[0][0] = "hello";
s2Array[0][1] = "Hello";
...
```

Elemente setzen

- Bsp. 2-D- Array:

```
String[][] s2Array = new String[5][3];
s2Array[0] = new String[4];
s2Array[1] = new String[2];
s2Array[2] = new String[3];
s2Array[3] = new String[4];
s2Array[4] = new String[4];
```

Aufbau: { {null, null, null, null}, {null, null} , {null, null, null}, ...}

Switch - Cases

WIEDERHOLUNG DER KONZEPTE

Beispiel

Folgende Datentypen funktionieren:
String, int, byte, short, char, Integer, Byte, Short, Character, Enums

- Kontrollstruktur, die eine Variable mit mehreren möglichen Werten vergleicht
- Syntax

Variable fruit hat den Wert „Apple“

```
String fruit = "Apple";

switch (fruit) {
   case "Apple":
      System.out.println("Dies ist ein Apfel.");
      break;
   case "Banana":
      System.out.println("Dies ist eine Banane.");
      break;
   default:
      System.out.println("Unbekannte Frucht.");
}
```

Variable wird übergeben

Mögliche Werte, die fruit annehmen kann

Signalisiert das Ende des switch-cases.
CONTINUE IST ILLEGAL!

Wenn keine case-Werte mit der Variable übereinstimmen

Ausgabe: Dies ist ein Apfel.

Beispiel

- Wenn das break fehlt, dann werden alle unterhalb liegenden Cases ebenfalls ausgeführt

```
String fruit = "Apple";

switch (fruit) {
    case "Apple":
        System.out.println("Dies ist ein Apfel.");
    case "Banana":
        System.out.println("Dies ist eine Banane.");
    default:
        System.out.println("Unbekannte Frucht.");
}
```

Ausgabe: Dies ist ein Apfel.
Dies ist eine Banane.
Unbekannte Frucht.

Beispiel

– Man kann cases auch verbinden, wenn sie alle das gleiche machen sollen

```
int number = 2;

switch (number) {
   case 1:
      System.out.println("Die Zahl ist zwischen 1 und 3.");
      break;
   case 2:
      System.out.println("Die Zahl ist zwischen 1 und 3.");
      break;
   case 3:
      System.out.println("Die Zahl ist zwischen 1 und 3.");
      break;
   default:
      System.out.println("Die Zahl ist außerhalb des Bereichs.");
}
```

Wiederholung für mehrere Cases

Beispiel

– Man kann cases auch verbinden, wenn sie alle das gl

```
int number = 2;

switch (number) {
   case 1:
      System.out.println("Die Zahl ist zwischen 1 und 3.");
      break;
   case 2:
      System.out.println("Die Zahl ist zwischen 1 und 3.");
      break;
   case 3:
      System.out.println("Die Zahl ist zwischen 1 und 3.");
      break;
   default:
      System.out.println("Die Zahl ist außerhalb des Bereichs.");
```

```
int number = 2;

switch (number) {
   case 1:
   case 2:
   case 3:
      System.out.println("Die Zahl ist zwischen 1 und 3.");
      break;
   default:
      System.out.println("Die Zahl ist außerhalb des Bereichs.");
}
```

Aufgabe

Schreibe ein Java-Programm, das den Monat anhand einer Zahl (1 bis 12) identifiziert. Die Zahl steht für den Monat im Jahr, und das Programm soll den Namen des Monats ausgeben.

- Wenn die Zahl von 1 bis 12 ist, soll der entsprechende Monat (z.B. 1 = Januar, 2 = Februar, ...) ausgegeben werden.
- Wenn die Zahl außerhalb des Bereichs liegt (z.B. 0 oder 13), soll "Ungültige Monatszahl" ausgegeben werden.

Loops, Loops, Loops

WIEDERHOLUNG DER KONZEPTE

While-Schleife

- Wiederholt einen Codeblock, solange eine Bedingung wahr ist
- die Bedingung auch irgendwann false ergibt, sonst Endlosschleife

```
int i = 1;
while (i <= 5) {
    System.out.println("Zahl: " + i);
    i++;
}
```

Do-While-Schleife

- führt den Code mindestens einmal aus, selbst wenn die Bedingung false ist!

```
int j = 1;
do {
    System.out.println("Wert von j: " + j);
    j++;
} while (j <= 5);
```

Do-While-Schleife

- Was ist die Ausgabe?

```
int j = 1;
do {
    System.out.println("Wert von j: " + j);
    j++;
} while (j >= 5);
```

For-Each-Schleife (enhanced for)

- wird zu Iteration durch Arrays oder Listen genutzt

- man kann nur von vorne nach hinten iterieren

```
int[] zahlen = {1, 2, 3, 4, 5};
for (int zahl : zahlen) {
    System.out.println("Zahl: " + zahl);
}
```

```
int[][] intArray = new int[][]{{1,2,3}, {1,2,3}, {1,2,3,4,5,6}};
for(int[] i : intArray){
    for(int j: i){
        System.out.println(j);
    }
}
```

For-Schleife

- für iterative Prozesse mit fester Anzahl an Wiederholungen
- man kann Elemente rückwärts ausgeben
- man kann Elemente alternierend ausgeben (Bsp.: jedes 2. Element ausgeben)

```
for (int k = 1; k <= 5; k+=2) {
   System.out.println("Durchlauf: " + k);
}
```

For-Schleife

- verschiedene Möglichkeiten zur Initialisierung

```
for (;;) {
    System.out.println("Durchlauf: " + k);
}
```

```
for(int x = 0;x<10;){
    System.out.println("Hello");
}
```

Endlosschleifen

```
int x = 0;
for(; x<10; x++){
    System.out.println("Hello");
}
```

Auslagern der Variable

For-Schleife (komplexer)

- die Bedingung in einer for-Schleife kann komplexer aussehen

```
for (int k = 1,  y = 0;  ((((k < 10) && (y-- > 2)) | k == 3)); k+=2) {
    System.out.println("Durchlauf: " + k);
}
```

For-

```java
public static void main(String[] args) {

    int[][] array = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9}
    };

    // Mit normaler for-Schleife auf die einzelnen Arrays zugreifen (außen)
    for (int i = 0; i < array.length; i++) {
        System.out.println("Zeile " + i + ":");

        // Mit for-each-Schleife auf die Elemente eines Arrays zugreifen (innen)
        for (int value : array[i]) {
            System.out.print(value + " ");
        }
        System.out.println();
    }

}
```

Break – Abbruch!

- break beendet die Schleife vollständig

```
for (int n = 1; n <= 10; n++) {
    if (n == 5) {
        System.out.println("Schleife gestoppt bei: " + n);
        break; // Beendet die Schleife komplett
    }
    System.out.println("Zahl: " + n);
}
```

Continue – nächster Schritt!

- continue springt zum nächsten Schleifendurchlauf

```
for (int m = 1; m <= 5; m++) {
    if (m == 3) {
        continue; // Springt über die 3 hinweg
    }
    System.out.println("Wert: " + m);
}
```

Aufgabe ca. 15 Minuten

- Schreibe eine while-Schleife, die die Zahlen von 10 bis 1 in absteigender Reihenfolge ausgibt.
- Erstelle eine do-while-Schleife, die den Benutzer nach einer Zahl zwischen 1 und 10 fragt. Falls der Wert außerhalb liegt, soll die Schleife erneut nach einer Zahl fragen.
- Schreibe eine for-Schleife, die die Summe der Zahlen von 1 bis 100 berechnet und ausgibt.
- Schreibe eine for-Schleife, die von 1 bis 10 zählt.
  - Falls die Zahl 6 erreicht wird, soll die Schleife mit break stoppen.
  - Falls die Zahl 3 ist, soll continue dafür sorgen, dass sie nicht ausgegeben wird.

Aufgabe ca. 15 Minuten

Gegeben ist folgender Array, der mit Namen gefüllt ist:

- String[] schueler = {"Anna", "Ben", "Clara", "David", "Emma"};

Gegeben ist ein weiterer Array, der mit Noten gefüllt ist:

- int[][] noten = { {2, 3, 1}, {4, 2, 5}, {1, 1, 2}, {3, 4, 2}, {2, 3, 3}};

Berechne und gib aus:

- Den Notendurchschnitt für **jeden Schüler** (Mittelwert seiner 3 Noten).
- Den **besten Schüler** mit der **niedrigsten Durchschnittsnote**.

```
Anna: Durchschnittsnote = 2.0
Ben: Durchschnittsnote = 3.67
Clara: Durchschnittsnote = 1.33
David: Durchschnittsnote = 3.0
Emma: Durchschnittsnote = 2.67

Bester Schüler: Clara mit einem Durchschnitt von 1.33
```